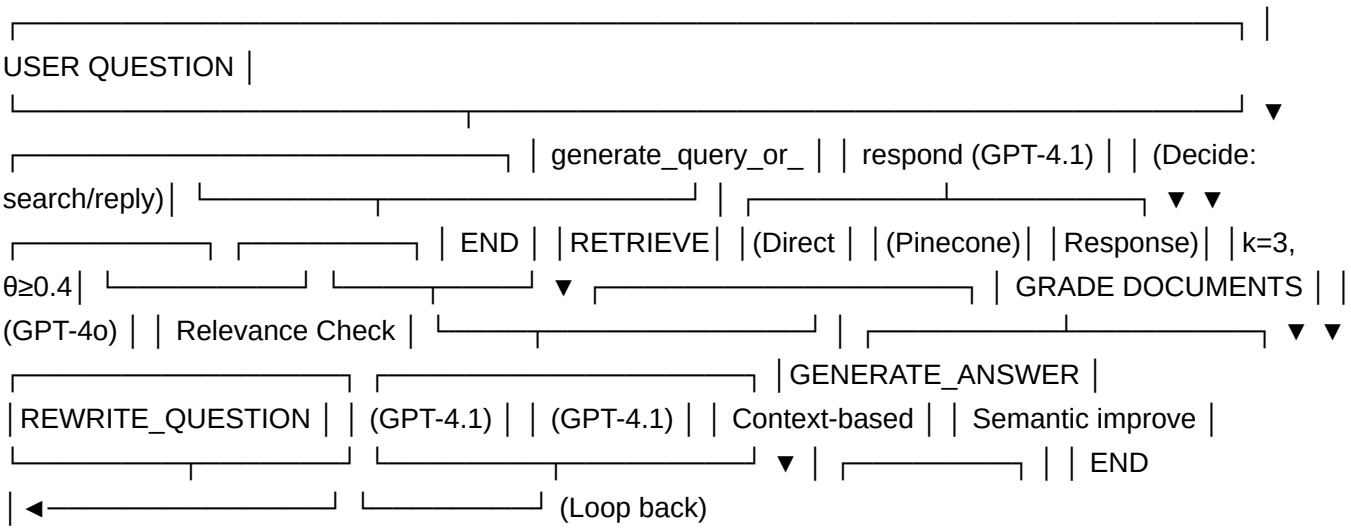
Hybrid RAG System

Tech stack

- uv (package manager)
- chonkie (chunking)
- pinecone (vector db)
- langchain (ai framework)
- fastapi (ai backend)
- langgraph (low level rag workflow)
- OpenAI (AI api)

Arsitektur RAG



Implementasi RAG System

Retriever Configuration (graph.py)

```
retriever = vector_store.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"k": 5, "score_threshold": 0.5},
)
```

Parameter:

- `k=5`: Mengambil 5 dokumen paling relevan
- `score_threshold=0.5`: Hanya dokumen dengan similarity ≥ 0.5

A. Adaptive Retrieval (graph.py:57-62)

Model **GPT-4.1** menentukan apakah perlu melakukan retrieval atau langsung menjawab:

- Jika pertanyaan butuh context dari knowledge base → gunakan retriever tool
- Jika pertanyaan umum/greeting → langsung respond

B. Document Grading (graph.py:82-93)

Model Grader: GPT-4o (temperature=0)

- Menilai relevansi dokumen yang di-retrieve
- Binary score: "yes" (relevant) atau "no" (not relevant)
- Jika relevant → generate answer
- Jika not relevant → rewrite question

C. Question Rewriting (graph.py:106-111)

Self-correction mechanism untuk improve query:

- Menganalisis semantic intent dari pertanyaan original
- Reformulasi pertanyaan untuk retrieval yang lebih baik
- Loop back ke `generate_query_or_respond`

D. Answer Generation (graph.py:124-130)

- Menggunakan retrieved context untuk generate jawaban
- Concise response (max 3 kalimat)
- Jujur jika tidak tahu jawaban

Document Processing Pipeline

Document Loading -> Markdon Chunking dengan Chonkie -> Metadata handling -> Upsert ke Pinecone

---

6. Testing & Evaluation

6.1 Testing Script (tracing-langsmith.py)

```
# Load test questions dari Excel
df_testing = pd.read_excel("Testing ChatAI Valak.xlsx", skiprows=1)
pertanyaan_list = df_testing.iloc[:, 3].dropna().tolist()

# Run batch testing dengan unique thread_id
for question in pertanyaan_list:
    thread_id = str(uuid.uuid4())
    final_state = graph.invoke(
        {"messages": [HumanMessage(content=question)]},
        config={"configurable": {"thread_id": thread_id}}
    )
```

Test Data:

- File: `Testing ChatAI Valak.xlsx`
- Format: Excel dengan kolom pertanyaan

10. Kesimpulan & Rekomendasi

10.1 Status RAG: ✅ IMPLEMENTED & FUNCTIONAL

RAG system sudah berfungsi dengan baik dengan fitur:

- ✅ Similarity search dengan threshold
- ✅ Document grading
- ✅ Question rewriting
- ✅ Self-correction loop
- ✅ 258 chunks ter-index di Pinecone

Next

- logging
- run evaluation
- semantic cache
- logging
- hybrid search

File Structure

```
chatbot-aktuaria/
├── graph.py                    # Main RAG workflow
├── tracing-langsmith.py        # Testing script
├── main-notebook.ipynb         # Data ingestion pipeline
├── langgraph.json              # LangGraph config
├── pyproject.toml              # Dependencies
├── .env                        # Environment variables
├── exported_doc.json           # Processed chunks
├── sample_docs/
│   ├── global_docs/            # 34 MD files
│   └── PANDUAN LENGKAP...      # Source docs
└── Testing ChatAI Valak.xlsx   # Test questions
```

Key Code Locations

- Retriever setup: `graph.py:45-53`
- Document grading: `graph.py:82-93`
- Question rewriting: `graph.py:106-111`
- Answer generation: `graph.py:124-130`
- Workflow definition: `graph.py:134-157`
- Document ingestion: `main-notebook.ipynb:cell[0bd5f312]`
